

CMPM 121 Assignment 4 Report

Architecture Diagram

```
relics.json
|
v
RelicLibrary.LoadAll()
|
v
RelicDefinition
|-- GetTriggerType()
|-- GetEffectType()
|-- GetDescription()
|-- Evaluate()
|-- GetString()
|
v
Relic
|-- Activate()
|-- Deactivate()
|-- Trigger()
|-- IsActive()
|
v
RelicFactory
|-- CreateTrigger()
|-- CreateEffect()

RelicTrigger implementations
|-- TakeDamageRelicTrigger.Register(), Unregister()
|-- EnemyKillRelicTrigger.Register(), Unregister()
|-- StandStillRelicTrigger.Register(), Unregister()
|-- WaveStartRelicTrigger.Register(), Unregister()
|-- WaveCompleteRelicTrigger.Register(), Unregister()
|-- MoveDistanceRelicTrigger.Register(), Unregister()

RelicEffect implementations
|-- GainManaRelicEffect.Apply(), Clear(), IsActive()
|-- GainSpellPowerRelicEffect.Apply(), Clear(), IsActive()
|-- GainMaxHealthRelicEffect.Apply(), Clear(), IsActive()

EventBus
|-- DoPlayerDamaged()
|-- DoEnemyKilled()
|-- DoSpellCast()
|-- DoPlayerMoved()
|-- DoPlayerMovementTick()
|-- DoWaveStarted()
|-- DoWaveCompleted()
|-- DoRelicPickedUp()
|-- DoRelicsCleared()

EnemySpawner
|-- LoadClassNames()
|-- SelectClass()
|-- StartLevel()
|-- NextWave()
|-- SpawnWave()
|
v
PlayerController
|-- StartLevel()
|-- AddRelic()
|-- BuildRelicRewardChoices()
```

```

| -- HasRelic()
| -- ApplyWaveStats()
| -- LoadClassConfig()
| -- EvaluateClassValue()
|
+-- reads classes.json

RewardScreenManager
| -- EnsurePendingReward()
| -- EnsurePendingRelicRewards()
| -- ShouldOfferRelicReward()
| -- GetRelicRewardDescription()
| -- CreateRelicButtons()
| -- UpdateRelicButtons()

RelicUIManager
| -- OnRelicPickedUp()
| -- ClearRelicViews()
RelicUI
| -- Apply()
| -- Refresh()

```

Architecture Description

The relic system is data-driven. RelicLibrary reads Resources/relics.json, converts each JSON object into a RelicDefinition, and creates runtime Relic objects from those definitions. A relic does not hard-code its behavior directly. Instead, RelicFactory builds one trigger object and one effect object from the trigger and effect type strings in the JSON.

Triggers are responsible for listening to gameplay events on EventBus. For example, the damage relic listens for OnPlayerDamaged, the kill relic listens for OnEnemyKilled, and the movement relic listens for OnPlayerMoved. When a trigger condition is satisfied, it calls Relic.Trigger(), which applies the relic effect to the owning PlayerController. Effects remain separate from triggers, so the same effect type can be reused by different conditions.

Temporary effects are handled by the effect object that created them. GainSpellPowerRelicEffect can add temporary spell power and remove it when a later event occurs, such as movement or the next spell cast. Immediate effects, such as gaining mana or increasing max health, apply once and then report that they are not active.

Relic rewards are integrated into the existing wave reward flow. RewardScreenManager offers relics after every third wave, asks PlayerController.BuildRelicRewardChoices() for three non-duplicate options, and stores those choices in GameManager. When the player advances to the next wave, EnemySpawner.NextWave() gives the selected relic to the player and activates it. RelicUIManager listens for relic pickup and clear events so the HUD stays synchronized with the player's relic list.

Character classes are also data-driven. EnemySpawner reads the class names from classes.json and adds class buttons to the start menu. The selected class is copied to PlayerController when a level starts. PlayerController.ApplyWaveStats() reevaluates health, mana, mana regeneration, spell power, and speed each wave using the class RPN expressions and the current wave number. Health and mana preserve their current percentage when maximum values change.

Added Relics

Ruby Heart: triggers when a wave is completed and increases the player's maximum health by 10. This uses the new wave-complete trigger and the new gain-max-health effect.

Wanderer's Coin: triggers after the player moves 40 units during a wave and restores 10 mana. This uses the new move-distance trigger with the existing mana gain effect.

Sun Dial: triggers when a new wave starts and gives the next spell 30 additional spell power. This uses the new wave-start trigger with the temporary spell power effect that clears after a spell is cast.

Contributions

Todd Crandell implemented the data-driven relic architecture, including RelicDefinition, Relic, RelicLibrary, trigger/effect factory creation, EventBus hooks, wave reward integration, class loading, class stat scaling, and the relic HUD. I learned that separating triggers from effects makes the system much easier to extend because new relics can be composed from small behavior pieces instead of becoming one-off branches.

Valerie Stratton expanded the player-facing systems that the relic work builds on, including the spell slot UI, slot selection, and drop controls from the previous assignment branch, then helped keep the reward and HUD flow understandable for players. I learned that gameplay state should stay in the controller/model layer while UI scripts reflect that state and forward player choices back into the system.